Fraud Detection Using Machine Learning

Ayush Manral

Capstone Project 2026

Project links:

Github Repository- https://github.com/ayushmanral10-crypto/Fraud_Detection_ML_capstone_project.git

Streamlit App link- https://frauddetectionmlcapstoneproject-nznayscn9kttgbsdeywyxq.streamlit.app/

Problem Statement

Financial institutions need an intelligent system that can:

- Detect fraudulent transactions
- Handle imbalanced datasets
- Reduce financial losses
- Work efficiently on real-world transaction data

•Missing value check

Dataset Overview

Dataset: transaction_fraud.csv

Target variable: *is_fraud*

- ***0 → Legitimate Transaction***
- ***1 → Fraud Transaction***

Performed:

- df.info()
- df.describe()
- Shape analysis
- Checking duplicate and missing values

Fraud Detection System

Traditional System

- Fixed Threshold per rule
- Limited to Yes/No decision
- Fail to capture interaction between features

ML models

- No fixed rule
- Provide probability score
- Capture interaction between features

Why ML for fraud detections?

- Better performance compared to rule-based systems.
- Can be combined with business rules.
- Adaptable to new fraud patterns.
- Scalable for large transaction volumes.

Class Imbalance

Checked using:

df['is_fraud'].value_counts()

- Observed highly imbalanced data and bias in nature, supporting majority 0 class.
- Used:

  class_weight='balanced'
  scale_pos_weight in XGBoost

Data Preprocessing

Steps:

- Separate feature and target variable

  x = df.drop('is_fraud', axis=1)
  y = df['is_fraud']

- Train and Test Split

  70% train, 30% test and stratified split

- Identified categorical columns

Feature Encoding

Used:

- *OnehotEncoder*
- *drop='first'*
- *handle_unknown='ignore'*

Implemented using: *ColumnTransformer*

Pipeline Implementation

- Created Pipeline:

```
Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('classifier', LogisticRegression())
])
```

ML Models Approached

Models:

- Logistic Regression
- Random Forest
- XGBoost

All models evaluated using:

- Accuracy
- Precision
- Recall
- F1 score
- ROC-AUC

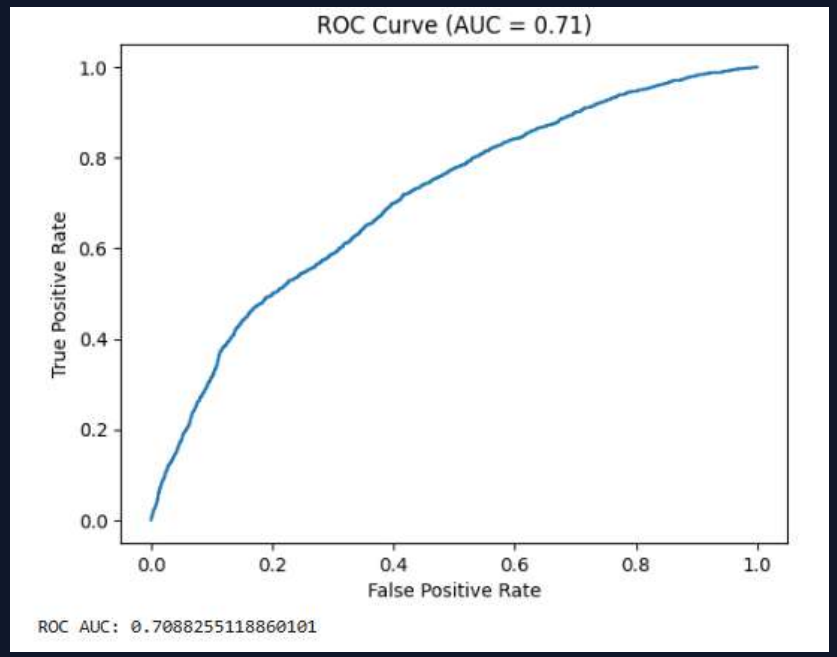
Model Comparison Table

Model	Precision	Recall	F1 score	Accuracy
Logistic Regression	0.22	0.51	0.30	0.76
Random Forest	0.29	0.18	0.22	0.87
XGBoost	0.11	0.96	0.20	0.25

Final Model Selection

- Why XGBoost:
  - XGBoost achieved: Highest recall 96%.
  - Good ROC-AUC score: 0.71.
  - Strong ability to detect minority fraud class even though the data is imbalanced.

	precision	recall	f1-score	support
0	0.97	0.17	0.29	13492
1	0.11	0.96	0.20	1508



Future Improvements

- Hyperparameter tuning:

  Apply GridSearchCV / RandomSearch
  Optimize learning_rate, max_depth, n_estimators
  Improve Recall and AUC performance

- Deploy using Streamlit

- Real time fraud detection API

Business Outcomes

- Reduced financial losses
- Improved fraud detection accuracy
- Enhanced customer trust
- Automated risk monitoring
- Scalable fraud prevention system

Final Conclusion

- Successfully built ML-based fraud detection system
- Handled imbalanced dataset
- Compared multiple models
- XGBoost performed best
- Evaluated using ROC & PR curves

....Thank You....